Context window	Number of Threads	Max Tokens Output	Run1	Run2	Run3	Run4	Run5	Median Time in Sec
128	1	128	2,185	2,390	2,312	2,590	2,215	**2,338**
256	1	128	2,597	2,593	2,123	2,449	2,601	**1,982**
512	1	128	2,626	2,433	2,224	2,258	2,340	**2,376**
1024	1	128	2,716	2,340	2,545	2,322	2,407	**2,466**
2048	1	128	2,707	2,708	2,194	2,228	2,426	**2,452**
8192	1	128	2,774	2,236	2,352	2,474	2,234	**2,414**
128	2	128	2,363	2,429	2,430	2,485	2,240	**2,389**
256	2	128	2,293	2,170	2,036	2,287	1,972	**2,151**
512	2	128	2,292	2,201	2,007	2,300	2,097	**2,179**
1024	2	128	2,395	2,151	2,381	2,378	2,041	**2,269**
2048	2	128	2,353	2,352	2,260	2,160	2,057	**2,236**
8192	2	128	2,561	2,555	2,196	2,298	2,083	**2,338**

Context window	Number of Threads	Max Tokens Output	Run1	Run2	Run3	Run4	Run5	Median Time in Sec
128	3	128	2,380	2,401	1,980	2,138	2,154	**2,210**
256	3	128	2,405	2,400	1,980	2,088	2,401	**2,254**
512	3	128	2,380	2,438	2,008	2,046	2,438	**2,262**
1024	3	128	2,423	2,352	2,498	2,294	2,141	**2,341**
2048	3	128	2,509	2,512	2,524	2,505	2,174	**2,444**
8192	3	128	2,546	2,543	2,491	2,088	2,186	**2,370**
128	4	128	2,408	2,188	2,061	2,014	1,996	**1,730**
256	4	128	2,427	2,208	2,157	2,008	1,985	**2,157**
512	4	128	2,468	2,214	2,180	2,044	2,026	**2,222**
1024	4	128	2,511	2,091	2,084	2,090	2,093	**2,173**
2048	4	128	2,415	2,395	2,099	2,187	2,268	**2,272**
8192	4	128	2,590	2,593	2,162	2,165	2,144	**2,330**

SmolLM2-135M Benchmark Report

Inference time vs. context window and thread count, 5 runs per configuration

1. Summary

The benchmark covers 24 configurations: 6 context windows (128 to 8192) by 4 thread counts (1 to 4), with output fixed at 128 tokens and 5 runs each (120 runs in total). Overall mean time is **2.30 s** per 128-token output, about **56 tokens/s**.

Key findings:

- **More threads help, but only modestly.** Going from 1 to 4 threads cuts mean time from 2.42 s to 2.21 s (about 9%). Most of that gain is already there at 2 threads (2.26 s). Threads 2, 3 and 4 are statistically hard to tell apart.
- **Context window has a small effect.** Mean time rises from about 2.27 s at 128 to about 2.36 s at 8192 (about 4%). The trend is consistent but weak.
- **Run-to-run noise is large.** The standard deviation within a configuration averages about 0.18 s (roughly 8%). That is bigger than most differences between neighbouring configurations.
- **The first run is systematically slower.** Run 1 averages 2.47 s versus about 2.20 s for run 5. This is a warm-up effect.
- **The "Median Time in Sec" column is not a median.** It contains the mean of the 5 runs, and 3 of the 24 values do not match even that (see section 5).

2. Results

Mean time in seconds over 5 runs, per configuration:

Context window	1 thread	2 threads	3 threads	4 threads
128	2,338	2,389	2,211	2,133
256	2,473	2,152	2,255	2,157
512	2,376	2,179	2,262	2,186
1024	2,466	2,269	2,342	2,174
2048	2,453	2,236	2,445	2,273
8192	2,414	2,339	2,371	2,331

Fastest configuration: 128 context, 4 threads (2.133 s). Slowest: 256 context, 1 thread (2.473 s). The full spread between best and worst mean is only about 0.34 s.

3. Analysis

3.1 Effect of thread count

Threads	Mean (s)	Median (s)	Std dev (s)

1	2,420	2,399	187
2	2,261	2,289	161
3	2,314	2,390	188
4	2,209	2,163	182

Each row pools 30 runs. Compared with 1 thread, 2 and 4 threads are faster by roughly 7% and 9%, which is above the noise level (standard error of about 0.03 to 0.04 s per row). The differences among 2, 3 and 4 threads are only 0.05 to 0.1 s, which is within noise. Three threads is not clearly worse than two, and four is not clearly better. In practice the model is small enough (135M parameters) that extra cores add little and the fixed per-run overhead dominates.

This revises the earlier single-run table, where 2 threads looked like a clear sweet spot and 4 threads looked slightly worse. With repeats, 4 threads comes out marginally best, which shows how unreliable single runs were.

3.2 Effect of context window

Context window	Mean (s)	Mean excluding run 1 (s)
128	2,268	2,251
256	2,259	2,216
512	2,251	2,203
1024	2,313	2,263
2048	2,352	2,316
8192	2,364	2,300

Times are flat up to 512, then rise slightly for 1024 and above. The total increase from 128 to 8192 is about 0.1 s. This is suggestive but not conclusive (about 1.7 standard errors). A plausible explanation is KV-cache allocation cost growing with window size, since the prompt is short and never fills the window. The benchmark therefore measures allocation overhead, not real attention cost at long context.

3.3 Warm-up and run-order effect

Run	1	2	3	4	5
Mean over all 24 configs (s)	2,472	2,371	2,220	2,246	2,197

Run 1 is at or above the row median in 22 of 24 configurations. Times drop by about 10% by run 3 and then level off. This points to cache and warm-up effects, or to a gradual change in system state during the session. Since all five runs of a configuration were taken back to back, this also inflates every configuration's mean and spread by a similar amount, so it does not bias comparisons much, but it does hide the true steady-state time (about 2.2 s).

3.4 Noise

The within-configuration standard deviation is 0.09 to 0.25 s, averaging 0.18 s. Individual runs range from 1.97 s to 2.77 s. Many apparent patterns in the table (for example 256 being slower than 512 at 1 thread) are therefore noise.

4. Conclusions

1. Use **2 or more threads**. Gains beyond 2 threads are not demonstrated by this data.
2. A **context window up to 512** costs nothing extra. Larger windows add a small amount, roughly 3 to 4%, at 2048 and 8192 with short prompts.
3. Expected time for 128 output tokens is **about 2.2 to 2.4 s** on this setup, regardless of configuration. The absolute figure is large relative to the model size, so a fixed overhead (model load, prompt handling or runtime setup) may be included in the timing.
4. Differences smaller than about 0.1 s should not be treated as real.

5. Data quality issues

- **Mislabelled column.** "Median Time in Sec" equals the arithmetic mean of Run1 to Run5 in 21 of 24 rows. The true medians differ by up to about 0.15 s (for example 256 context, 1 thread: median 2.593 s versus mean 2.473 s).
- **Three inconsistent values.** 256 context with 1 thread shows 1.982 s (the five runs average 2.473 s). 128 context with 4 threads shows 1.730 s (runs average 2.133 s). 512 context with 4 threads shows 2.222 s (runs average 2.186 s). None of these can be reproduced from the listed runs, and the first two are lower than any single run in their row. They look like stale or hand-edited values and should be fixed.
- **Missing setup information.** The file does not state hardware, runtime, quantization, prompt length, whether model load is inside the timed window, or whether output always reached 128 tokens.